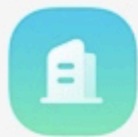


US使用GBF建设portal实践

一、背景



新房



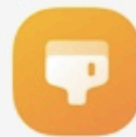
二手房



租房



商业办公



装修

热门新盘榜

精选小区

办公选址



紫金书院

昌平区 建面 111-146m²

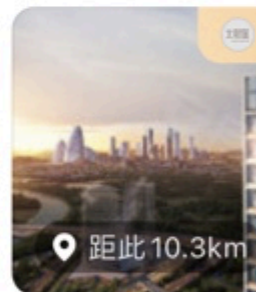
暂无



宸悦国际

昌平区 建面 57-121m²

291万/套起



京投发展·北苑

朝阳区 建面 80-

暂无

小区

新房

二手房

租房

租写字楼

售写

区域 ▾

价格 ▾

房型 ▾

更多 ▾



甄选

福熙南区，首付低，单价低（13号线北苑）（业主已定房）可谈价

3室2厅 162.92m² 南北

楼龄新 绿化率高

1388万 8.52万/m²

立即咨询

位于“福熙大道” 距此2.6公里



甄选

南北通透，一手毛坯，118平诚心出



驰名地产

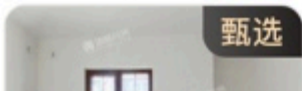
售

2室2厅 118m² 南北

南北通透

940万 7.97万/m²

位于“林奥嘉园”距此2.7公里



甄选

法式石材外立面独栋，前二后三毛
杯一样板间示范区 正南入户

11:27



< 酒店



玉渊潭公园

牛街

情侣酒店

设置我喜欢的星级/价格



查找酒店

7x24h服务 退订便捷 全网比价 全地图覆盖

精品民宿

品质甄选



钟点房

高性价比



商旅返现

单单必到账



领券中心

抢5折券



五一提前订 早订9折起



北京望京希尔顿欢朋酒店

8折起 ¥518起 ¥630

今日爆款 人气酒店限时抢



花美时民宿 (北京朝阳鸟巢大...)

8折起 ¥405起 ¥479



热门推荐

城市精品



附近热卖

步行1公里



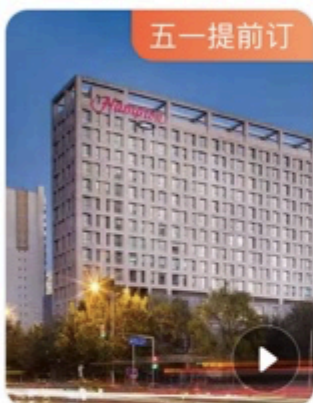
影音房

视听盛宴



情侣约会

舒适浪漫



五一提前订

北京望京希尔顿欢朋酒店 高档型

4.8非常棒 60评价

距你 3.9 公里 凯德 MALL(望京店) 附近

“房间很大，视野很开阔楼层很高”

高德指南 高档型酒店 第3名 商务出行 近地铁

58分钟前有人购买

¥630 ¥518起

高德红包 | 立减 | 代惠 112



五一提前订

锦江之星 (北京会议中心鸟巢)

横琴
欢乐购

经济型

4.4很好 1103评价

距你 17 米 来广营

“距离北京国家会议中心比较近”

高德指南 经济连锁酒店 第4名 商务出行 叫醒服务



订单

¥275 **¥263**起

10:05

高德地图



< 优惠加油



硬核补贴钜给力
新人加油礼包

立即领取

新

新人专享

免

调价补贴



领券中心



大额油券



车主服务

最高赢200元免单大奖

踏春有约 趣加油

立即点击>



新用户首单

满200减18

满200减9

领取>

推荐排序 ▾

92# ▾

全部品牌 ▾



优惠加油

庆轩加油站 (来广营西路)

3km 最便宜

1.2 公里 朝阳区来广营西路南侧

“站内会提供免费的零食”

¥6.83/L 加200省16 | 可用券 油站价 ¥7.44/L

到油站后输入本次加油金额

立即加油



优惠加油

中化石油吉利北苑加油站

1.8 公里 朝阳区北苑路17号

加200省8 | 可用券

¥18

¥7.15/L 油站价 ¥7.44/L

立即

领



中油随时广开源加油站

4.4 公里 朝阳区北四环东路辅路

“加油后免费洗车位置也好找”



订单

¥6.73/L 加200省19 | 可用券
油站价 ¥7.44/L

立即加油

< 超市/购物

Q 搜索商户



商场



超市



服装



步行街



便利店

优选商场



新世界百货(望京店)

4.8 超棒 商场 38条评价 半年销量1

距你 5.6 公里 望京南(地铁站)附近

“还有阿姨在门口跳广场舞，有夜市...”



推荐 商场 超市便利 步行街



物美超市(北苑东店)

★★★★★ 4.8 18条

来广营 综合超市

距你 432m

高德指南 来广营超市人气榜 第1名



盒马鲜生(顾家庄桥店)

★★★★★ 4.8 19条

来广营 生鲜超市

距你 299m

高德指南 来广营超市人气榜 第3名

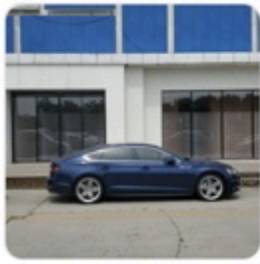


京百万超市(来广营店)

★★★★☆ 4.2 2条

来广营 超市

距你 896m



Frandiss

★★★★☆ 3.8

来广营 服饰鞋包

距你 289m



来广营 服饰鞋包

10:03

高德地图



< 休闲娱乐

北京市 ▼ 搜索商户/玩乐项目等



足疗



洗浴



KTV



按摩



SPA



养生馆



网吧



茶座



垂钓园



更多

高德红包 满50减3

领取 >

推荐

网吧

按摩

洗浴

KTV

酒吧

垂钓园



华夏良子 (后舍店)

4.7 超棒 足疗 半年销量8466

距你 5.5 公里 新世界百货 (望京店) 附近

“手法相当老道，以后还会再来”

高德指南 望京按摩/足疗 第6名 免费茶点 精油SPA 影院式



【优】馨悦阁 SPA会所 (芍药居北里店)

4.2 推荐 SPA 半年销量326

距你 4.3 公里 远洋·未来广场购物中心附近

“服务热情，环境优雅，身心愉悦”

高德指南 太阳宫村按摩/足疗 第2名



春秋茶楼·棋牌 (北苑店)

4.7 超棒 棋牌室 ¥61/人 半年销量50

距你 2.9 公里 易事达广场附近

“特别划算了，小自习室一个人也得70的”



【优】水·润 SPA (天创世缘店)



4.2 推荐 SPA 半年销量11

距你 3.6 公里 阳光广场附近

 “环境优雅大气上档次手法专业”

日式风



尚古足道 (北苑店)

4.5 超棒 足疗 ¥148/人 半年销量2608

距你 2.0 公里 易事达广场附近

好评如潮 专业技师 环境优雅 价格公道 技师手法到位



10:03

高德地图



< 旅游

北京市 Q 景点

写评论赢30元门票红包

大牌景区等你来点评

GO >



自驾周边



著名景区



赏花踏青



公园



动物园

优惠专区

¥3.9 抢 ¥45



订火车票

机票客车票



周边游

周边好去处



出行保

自驾旅行险



当季好去处

热门目的地

踏青赏花正当时

逃离都市拥抱自然

短途自驾周边游

当



玉渊潭公园

赏樱胜地 3月中-5月下旬



圆明园遗址公园

被誉为“一切园林艺术...



北海公园

春日带娃体验

限时抢购



北京环球度假区-1日门票

¥488起 ¥519 9.4折

免费游览



北京温榆河公园(朝阳...

免费



目前us有 美食、酒店、旅游、休娱、房产、商超、火客飞、周边游等这些行业的portal落地页，每个portal页的维护迭代不论是从产品、开发还是组织、大部分代码实现都是完全隔离的。



这种隔离方式导致各业务之间存在信息茧房，唯一的交集是代码库，各组开发同学在没有portal页顶层设计输入的情况下，难有业务全局观，不同portal页各自开发，相同模块在没有统一设计和管理的情况下面临两种问题：



复制代码

- 方案链路不一致，多种链路交替，各自维护
- 存在重复逻辑，散落在不同应用中，代码各自为战，重复实现

在方案不统一、重复建设的情况下，us的portal面临着更长期的问题：



复制代码

- 系统设计的质量不高，故障率和风险加大
- 开发维护成本大致随着portal页的数量线性增长
- 无知识管理、团队之间协作困难，容易产生信息孤岛和理解不一致
- 容易产生系统之间边界不清晰，加大维护成本

二、目标

通过统一的、规范的、合理的系统设计做到增强复用、降本提效，提高稳定性



复制代码

- 对明确的固定场景，有统一的、稳定的实现方案
- 提高设计质量和可维护性，降低故障率和风险
- 减少重复工作，降低开发维护成本
- 提高知识管理，促进团队协作，避免信息孤岛和误解
- 使不同团队、系统之间的边界清晰，减少内耗

三、设计

1、为什么要用GBF

GBF使用了**领域**的概念：在研究某个问题时，将问题划分为若干个相互独立的子问题，以便更好地组织、分析和解决问题。领域划分的作用在US的体现主要有以下几点：



复制代码

- **提高问题分析的精度**：使用划定好的领域，能够快速将需求点定位到某个具体的问题域，从而将某一类问题归类管理，简化复杂性的同时也能够让领域内部的逻辑能更聚焦
- **促进知识共享和演进**：使用领域构建系统后，某个具体的领域交给专人维护，培养领域专家，做好领域模型表达促进技术和产品之间的交流和合作，促进知识共享，高效支持业务分析。
- **提高复用性，降低开发成本**：可以将相似的功能模块归为同一领域，使得不同业务能力可以共享同一领域的逻辑，提高代码的复用性。不同的开发小组可以同时并行开发各自的领域，避免小组之间的冲突和耦合问题，提高开发效率

- **提高稳定性**: 划分领域可以将系统分解成多个部分，使得每个部分的功能更加独立，发生问题时不会对整个系统产生影响，提高整体的稳定性

拿个具体的例子来看：各行业portal页的金刚位。目前us的portal页金刚位不同场景、行业均有不同的实现方式，如果涉及到不同应用相似逻辑容易有cv代码的情况

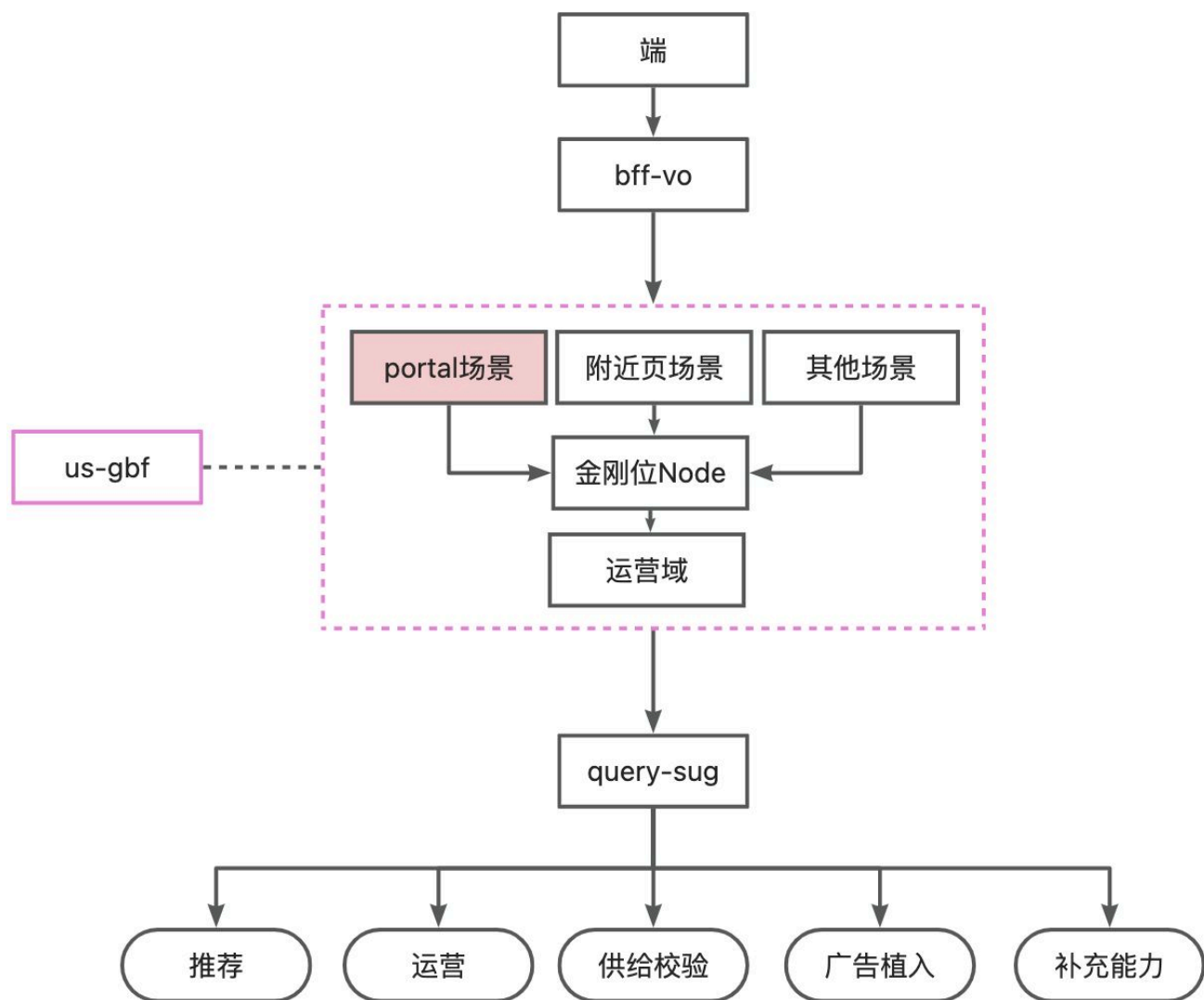


复制代码

- 景区、周边游之前取的diamond，后来切到了bff-other
- 休娱portal页取的merger-sp
- 房产portal取的house-sp
- 其他场景例如附近页取的bast，us内部通过diamond有干预逻辑

通过现状我们看到每个行业或场景的金刚位实现方案都是不一样的，不同的小组开发、不同的链路方案，意味着迭代维护成本居高不下，虽然目前的新方案bff-other统一对金刚位进行了收拢，但是只对输出进行了统一，底层的逻辑差异还在，且不同逻辑的实现无统一标准管理，不方便复用。

基于领域的系统建设，金刚位是一个具体的问题，适合放在一个地方解决实现。也正是基于**领域建设**这个抓手，将金刚位这个功能点统一组织、分析，拆解金刚位需要提供哪些能力，进而逻辑分类、链路归一，然后沉淀出金刚位统一的标准链路方案。



链路方案归一后，就可以统一输入和输出



复制代码

```

bizId          身份id，代表不同行业（hotel、scenic、house、unkonw）。
scenario        场景id，例如 portal（落地页）、nearyby(附近页)、search(搜索)。
// 位置参数：依据推荐位id选择提供必要位置参数
user_loc        用户定位（x，y坐标）
geoobj          图面信息（x1,y1;x2,y2)portal页场景一般不会有这个参数。
cityAdcode      城市adcode（4位有效数字，后两位抹0，直辖市后四位抹0）。
// 透传公参
diu、adiu、uid、gsid、csid、usid、div、superid、stepid、tid、testid
ajaxVersion     前端bundle版控
traceAppId      应用id
tracePageId     页面id
traceModuleId   模块id
sourcefrom      压测标 loadrunnertest。
personal_switch 个性化开关
  
```

```

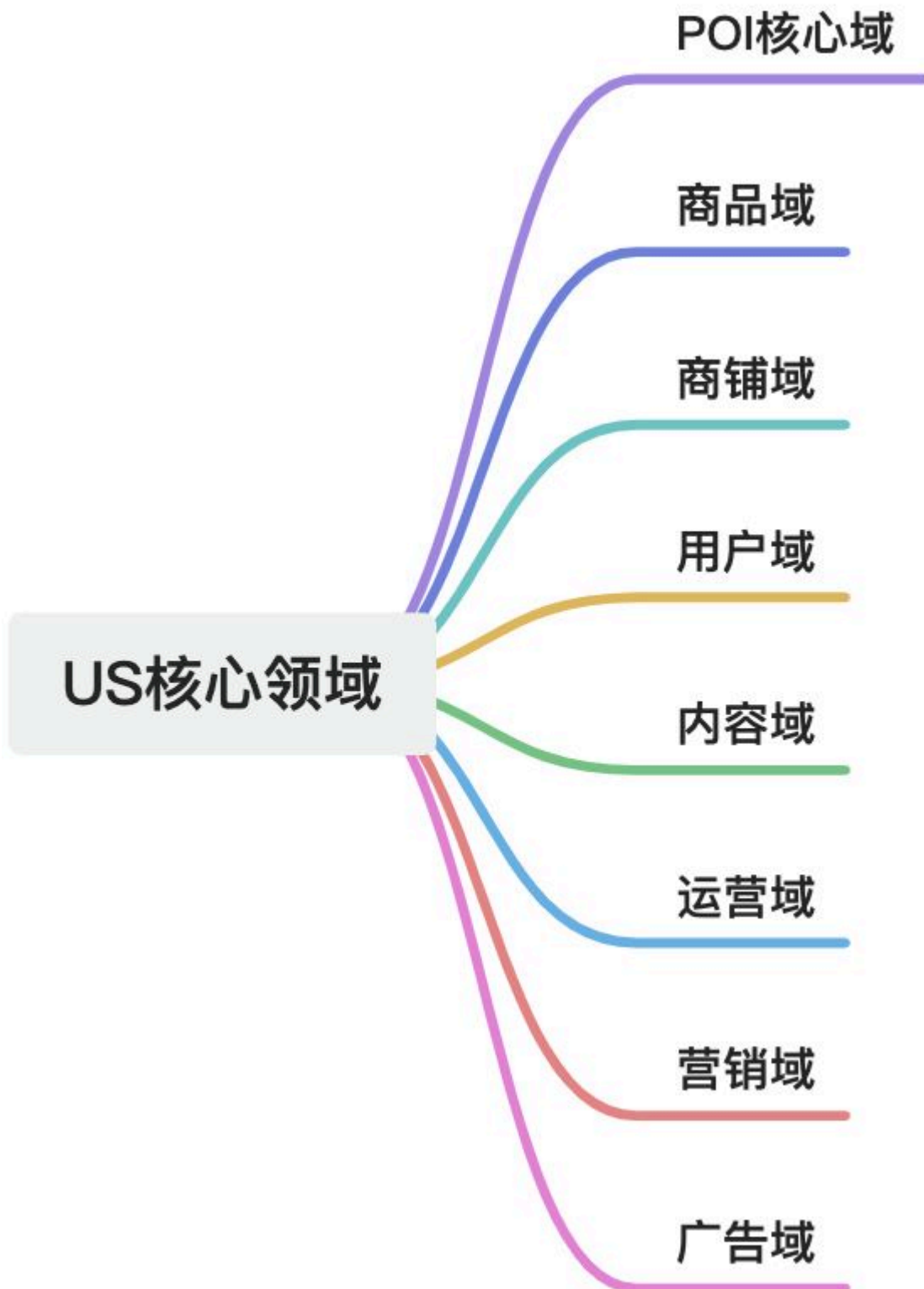
{
  "code": 1,
  "message": "success",
  "data": {
    "items": [
      {
        "icon": "http://", //icon地址
        "icon_height": 128,
        "icon_width": 56,
        "title": "海洋馆", //展示标题文案
        "tool_tips": "9.9元", //icon右上角的气泡
        "log_info": {}, //上报埋点信息，需要算法产生的无业务语义只用来统计指标的埋点在这里存放，如果业务字段中已存在的这里不再冗余
        "schema": "amapuri://" //跳转schema, TODO 看是querysug拼还是us拼，有可能会有动态的传参，例如adcode
                                amapuri://search/around?
                                city=${adcode}&lat=${y}&lon=${x} 入参us传，querysug替换占位符生成schema
      }
    ]
  }
}

```

至此我们看到了us对金刚位的实现包括统一的链路、输入输出、逻辑管理，接近理想态的实现，无论是从复用性、开发成本、需求方案对接成本能够有明显的提升，稳定性建设也可针对某一具体的领域或功能点开展。

2、使用GBF概念设计

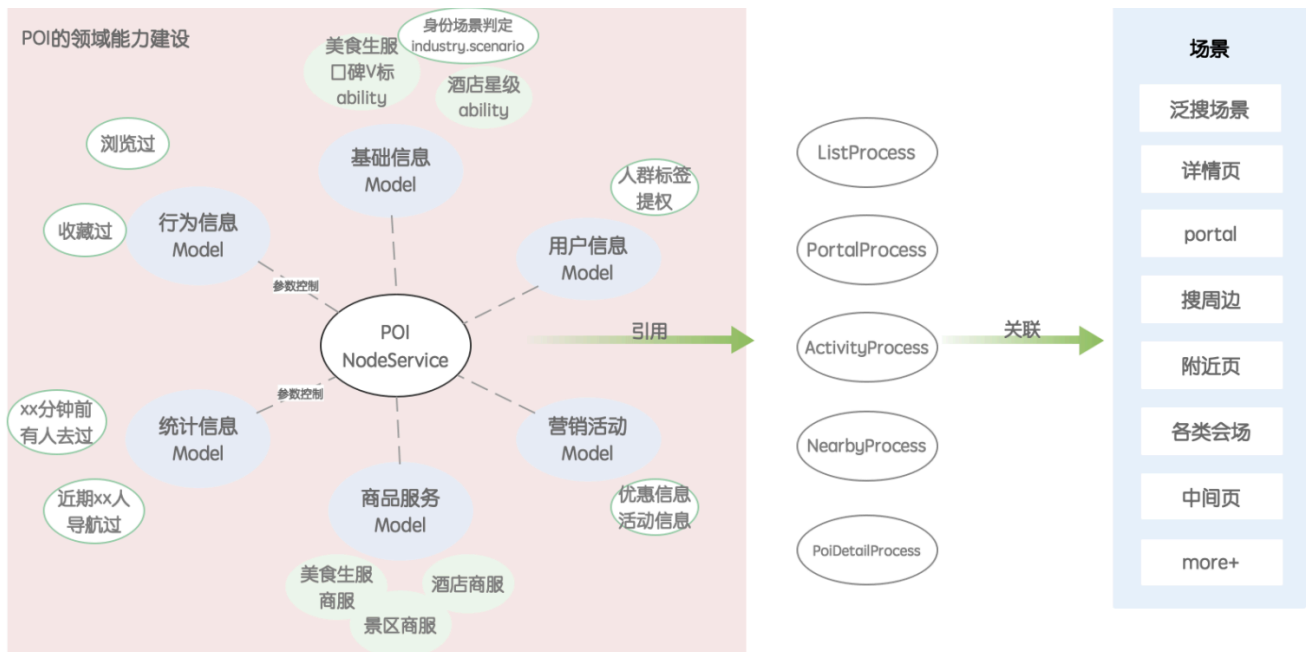
① 全局领域划分



- 目前US在各个域内的逻辑相对复杂度不高，为避免过度设计，按照指导原则进行聚合设计出来8个领域
- 支持在后续领域复杂度提升支持进行域的拆分

参考域建设指导原则：[US业务域梳理及指导原则](#)

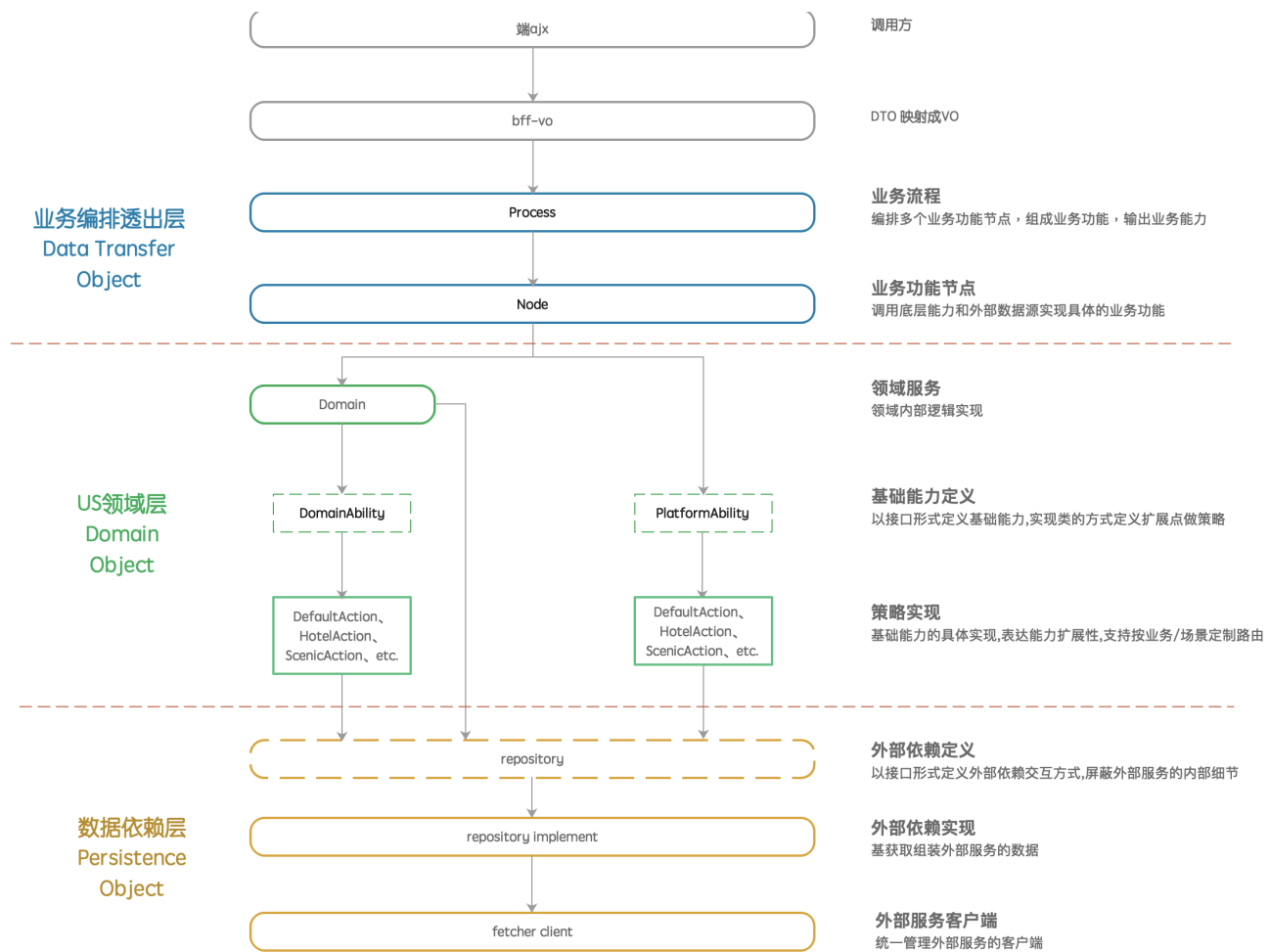
② 统一领域建设



US以前抽象和复用的角度主要关注点在链路上，缺少领域的管理，容易导致当链路发生变化时，各领域间的调用和依赖关系发生调整，协议也容易发生变化（系统模块间解耦不够清晰），不同应用之间出现逻辑相似度高但无法复用的代码。拿POI卡这个例子看，当抽象管理的维度添加领域对象后，可以归纳出如上图的结构关系，借此可以明确各领域的数据结构表达和管理规范每个领域之间的行为。当POINodeService明确后，标准业务能力层的Process可以直接引用，两者之间只剩下引用关系，不再有任何的关联关系。Process对应着不同的场景，自此POI的领域模型和方法对标准能力和场景解耦。

③ 系统分层，标准输入输出

gbf应用分层



gbf对象分层

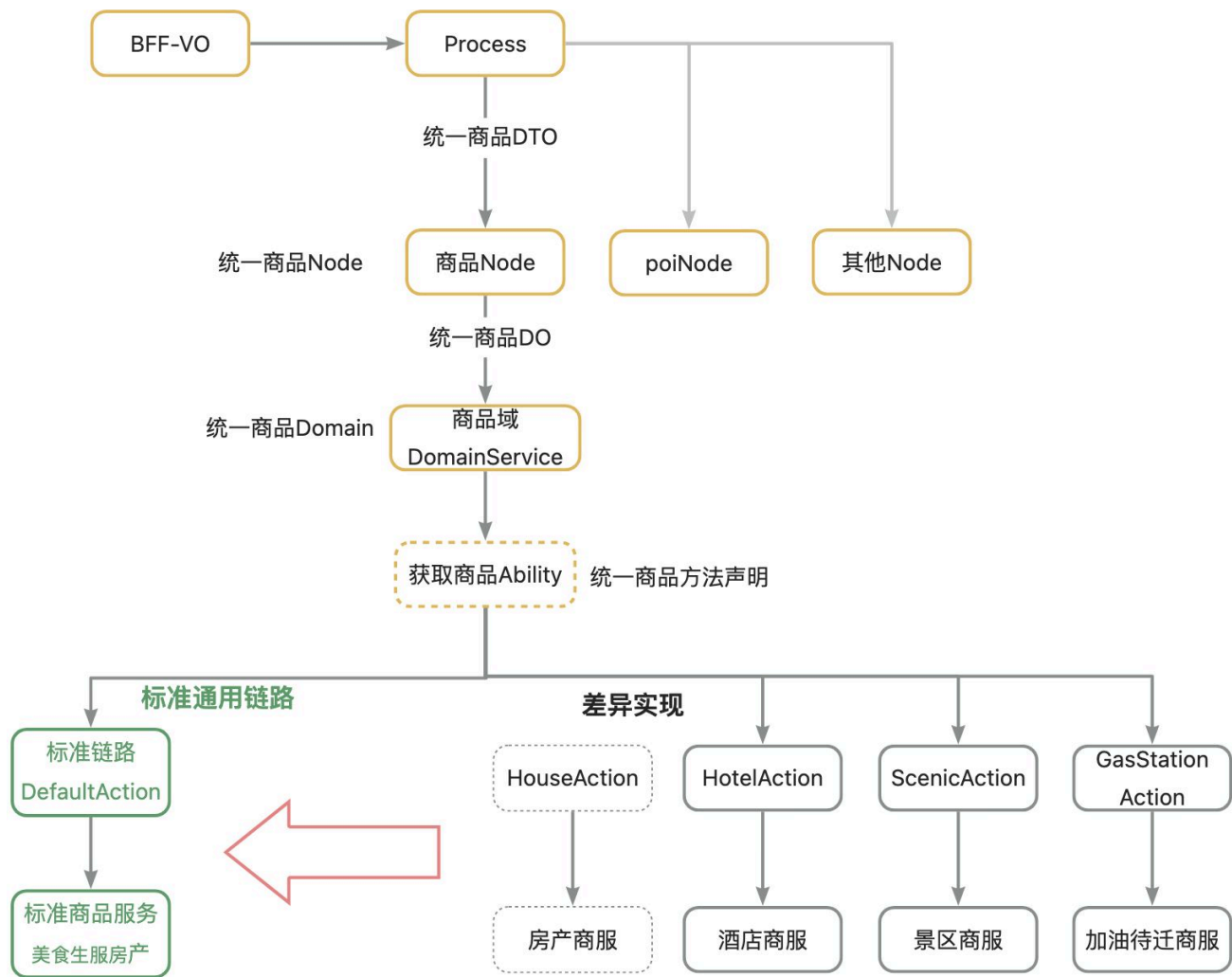
前端展现
VO

业务能力
DTO

领域模型
DO

数据来源
PO

④接口行为统一，差异定制实现

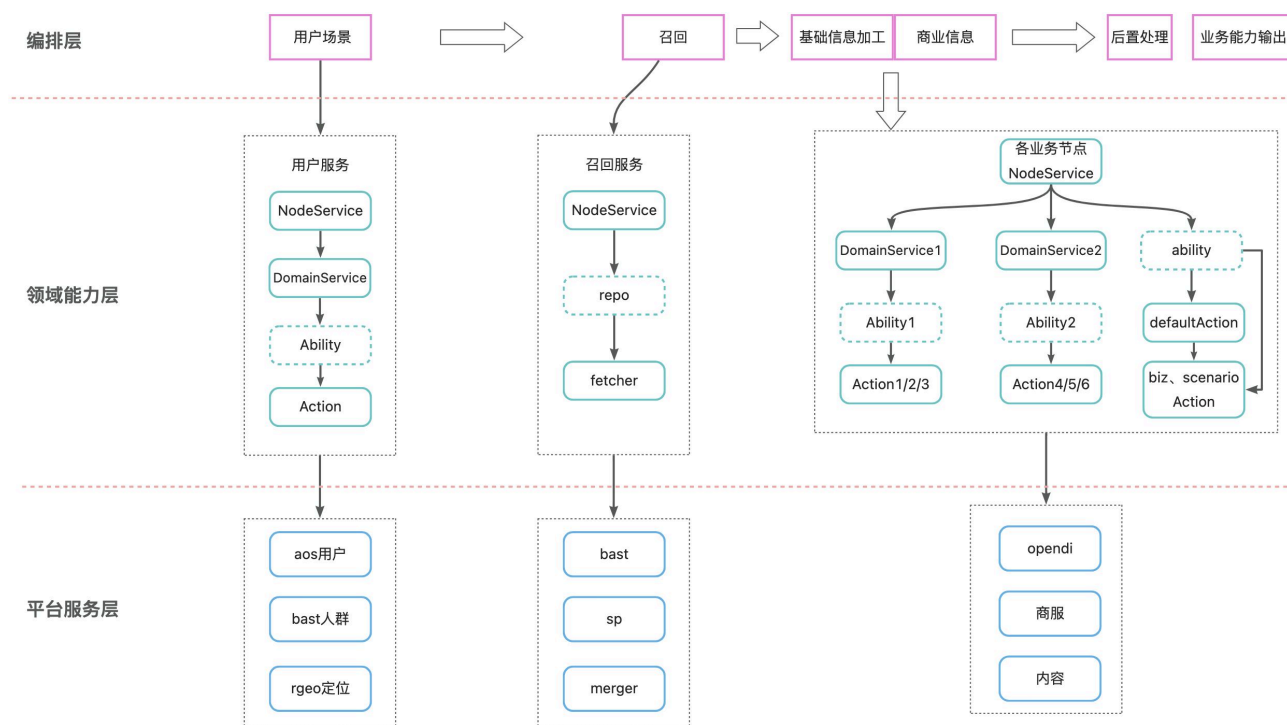


⑤portal整体架构

结构



流程



3、使用GBF开发

使用GBF实现一个简单逻辑，以us的金刚位调用quersug下游为例

1、在repository包里定义fetcher的输入reqParam、输出 PO，在repo impl中创建声明式fetcher

```
@FetcherClient(name = "sug-fetcher")
public interface SugFetcher {
    @FetcherMethod(name = "quicklink")
    QuerySugResult<QuickLinkPO> getQuickLink(@RequestParam
    QuickLinkReqParam reqParam);
}
```

对接口服务的调用使用声明的方式编写代码，能够直观表达接口参数和协议的同时还可以借助业脉平台方便的批量导出文档。

2、在repository中定义repo接口，在impl中定义实现

```
public interface QuickLinkRepo {

    @Annotation(title = "获取金刚位")
```

```

        QuickLinkPO getQuickLink(QuickLinkReqParam reqParam);
    }

@Service
public class QuickLinkRepoImpl implements QuickLinkRepo {
    @Resource
    SugFetcher sugFetcher;
    @Override
    public QuickLinkPO getQuickLink(QuickLinkReqParam reqParam) {
        return Optional.ofNullable(sugFetcher.getQuickLink(reqParam))
            .map(QuerySugResult::getData)
            .orElse(null);
    }
}

```

多这一层的目的是为了简单处理fetcher返回的结果，例如poi的merge逻辑、特殊情况下对fetcher实现缓存等操作

3、定义金刚位的DO和入参（这里入参没有特殊参数直接使用了基类），定义金刚位的DomainService接口，和对应的实现类

[复制代码](#)

```

@DomainService(name = "运营领域金刚位服务", domain =
OperationDomain.class)
public interface IQuickLinkDomainService {
    QuickLinkDO getQuickLink(BaseRequest baseRequest);
}

@Service
public class QuickLinkDomainService implements IQuickLinkDomainService {

    @Resource
    QuickLinkRepo quickLinkRepo;

    @Resource
    IQuickLinkAbility quickLinkAbility;

    @Override
    public QuickLinkDO getQuickLink(BaseRequest baseRequest) {
        //构建请求参数
        final QuickLinkReqParam quickLinkReqParam =
quickLinkAbility.buildQueryParam(baseRequest);
        //请求金刚位服务
    }
}

```

```

        final QuickLinkPO quickLinkPO =
quickLinkRepo.getQuickLink(quickLinkReqParam);
        //根据返回结果构建金刚位DO（标准DO）可作为DTO输出
        final quickLinkDO quickLinkDO =
quickLinkAbility.buildQuickLinkDO(quickLinkPO);

        return quickLinkDO;
    }
}

```

抽象领域方法内的行为，生成金刚位的三部曲：构建请求参数、请求金刚位服务、构建金刚位DO返回结果。

这块是需要注意的地方，为什么要做接口的抽象？因为要做行为的管理，行为管理的意义分多个方面去看：一方面在于统一输入输出，将流程的代码逻辑放到实现类里，DomainService里的领域方法就可以比较清晰的表达出来；

另一方面接口抽象后就可以有不同的实现方式，这就和设计模式里的策略模式一样，GBF使用了bizId(业务身份，代表行业) scenario（场景id）作为策略的key，对应不同的行业就可以有不同的实现类，从而做到实现差异化的同时又能够统一输入和输出。尽管很多时候现存的差异化从长期来看没有存在的意义，作为短中期的实现这是比较理想的方案。

4、定义ability、实现action

复制代码

```

@DomainAbility(name = "金刚位ability", domain = OperationDomain.class)
public interface IQuickLinkAbility {

    @ActionExtensible(name = "构建金刚位请求参数")
    QuickLinkReqParam buildQueryParam(BaseRequest request);

    @ActionExtensible(name = "构建金刚位DO")
    QuickLinkDO buildKQuickLinkDO(QuickLinkPO quickLinkPO);

}

```

复制代码

```

@Component
public class DefaultQuickLinkAction implements IQuickLinkAbility {
    @Override
    @Action(name = "构建金刚位请求参数")
    public QuickLinkReqParam buildQueryParam(BaseRequest request) {
        return new QuickLinkReqParam(request);
    }
}

```

```

@Override
@Action(name = "构建金刚位DO")
public QuickLinkDO buildQuickLinkDO(QuickLinkPO quickLinkPO) {
    return
Optional.ofNullable(quickLinkPO).map(QuickLinkPO::getItems)
    .map(Collection::stream)
    .map(itemStream -> itemStream.map(po ->
QuickLinkDO.ItemDO.builder()
        .title(po.getTitle())
        .icon(po.getIcon())
        .iconHeight(po.getIconHeight())
        .iconWidth(po.getIconWidth())
        .logInfo(po.getLogInfo())
        .toolTips(po.getToolTips())
        .schema(po.getSchema())
        .build()).collect(Collectors.toList()))
    .map(QuickLinkDO::new).orElse(null);
}
}

```

这一步的ability定义和上一步的domain方法是紧密相关的，ability定义后一般会有一个默认实现，如果不同场景和行业有差异（例如hotel）就需要在us-app-hotel这个工程里写对应的Action实现ability。GBF底层是通过包名的方式（com.amap.us.app.hotel.action）结合application.yaml的配置(biz-id映射的package)去路由到对应的Action实现，场景这块则是根据@Scenario注解来作区分，这也是GBF基于业务身份和场景做策略的实现方式。

5、实现NodeService

复制代码

```

@ProcessNode(name = "QuickLinkService")
public class QuickLinkNodeService extends
AbstractCommonNodeService<QuickLinkDO, BaseRequest>{
    @Autowired
    IQuickLinkDomainService quickLinkDomainService;

    @Override
    public QuickLinkDO doInvoke(BaseRequest request) {
        return quickLinkDomainService.getQuickLink(request);
    }
}

```

Node的定位可参照GBF的相关资料，这里逻辑比较简单，也会存在逻辑复杂的场景，例如走merger链路的feed流，走营销获取活动信息盘货信息再走merger找回的运营位。

6、Process注册Node

▼

复制代码

```
//在ProcessConfig中声明Node
BaseNodeConfigBuilder quickLinkNode = new NodeConfigBuilder()
    .preJoinPoint(new JoinPoint<PortalParam, BaseRequest>() {
        @Override
        public BaseRequest join(PortalParam request) {
            return request;
        }
    })
    .nodeBeanName(BeanUtil.getBeanName(QuickLinkNodeService.class));

//processConfig注册Node
@Bean
public ProcessConfig portalProcess() {
    return new ProcessConfigBuilder()
        .request(Request.class)
        .response(CommonResponse.class)
        .nodes(userNode)
        .nodes(filterTabNode)
        .nodes(feedNode)
        .nodes(bannerNode)
        .nodes(quickLinkNode)
        .build();
}
```

7、接口注册

▼

复制代码

```
@RestController
@RequestMapping("/process")
public class ProcessController {

    @GetMapping("/portal/{biz}")
    @ResponseBody
    public Result<Response> portal(@PathVariable String biz,
        PortalParam param) {
        NodeService<Response, Request> process =
            BeanUtil.getSpringBean(ProcessRegister.genericBeanName(PortalProcessConfig.BEAN_NAME));
    }
}
```

```
param.setBizId(biz);
return process == null ? ProcessResult.fail("未找到流程") :
process.invoke(param);
}
}
```

至此使用GBF实现一个简单完整的流程就完成了。

四、总结

基于领域建设的思想，使用GBF构建业务能力能够给US带来以下几点优势：

- 帮助us**沉淀**各领域的业务，**标准化**流程链路，明确业务逻辑
- 需求拆解后能够有对应的领域收拢，各领域间**无耦合**，使问题更**聚焦**
- 领域沉淀的能力方法能够很好的复用，不同场景间的功能节点基于领域能力去构建，构建好的能力可以在其他场景（附近页、会场、泛搜等）直接**复用**，提高开发**效率**
- 进一步**强化**能力逻辑的**表达**，借助业脉系统可视化流程构建专业**知识库**，帮助开发同学快速、直观的了解业务，进而提高技术对业务的scense，有全局观的同时又能够方便把握细节，提高产品和技术、技术和技术之间的**沟通效率**
- 可针对某个领域或功能节点进行**稳定性**建设，减少不同功能节点之间的依赖，提高整体稳定性

业脉对业务流程的表达

业脉系统

日常环境

XXX 平台

技术交流群: 12345678

用户名

指标大盘

业务流程

接口测试

平台层

流程-Process

节点-Node

能力-Ability

默认实现-Action

领域层

领域-Domain

域服务-DomainService

能力-Ability

默认实现-Action

扩展层

业务实现-Action

管理

平台管理

应用管理

XXX 页面标题

应用: 请选择

类型: 请选择

① 点击节点跳转到详情页

流程xxx (processxxx) 详情

流程xxx (processxxx) 详情

流程xxx (processxxx) 详情

流程xxx (processxxx) 详情

流程xxx (processxxx) 详情

流程xxx (processxxx) 详情

流程xxx (processxxx) 详情

Node1 (普通节点)

Process1 (开发流程)

Process2 (普通流程)

Node2 (聚合节点)

Process3 (流式流程)

Node3 (普通节点)

平台扩展

领域扩展

节点: 全部

1 / 5

编码	名称	描述	节点	操作
platformAbility1	平台扩展1	能力描述	Node1	默认 扩展
platformAbility1	平台扩展1	能力描述	Node2	默认 扩展

业脉系统

日常环境

XXX 平台

技术交流群: 12345678

用户名

指标大盘

业务流程

接口测试

平台层

流程-Process

节点-Node

能力-Ability

默认实现-Action

领域层

领域-Domain

域服务-DomainService

能力-Ability

默认实现-Action

扩展层

业务实现-Action

管理

平台管理

应用管理

XXX 页面标题

应用: 请选择

类型: 请选择

① 点击节点跳转到详情页

流程xxx (processxxx) 详情

流程xxx (processxxx) 详情

流程xxx (processxxx) 详情

流程xxx (processxxx) 详情

流程xxx (processxxx) 详情

流程xxx (processxxx) 详情

流程xxx (processxxx) 详情

Node1 (普通节点)

Process1 (开发流程)

Process2 (普通流程)

Node4 (普通节点)

Process4 (普通流程)

Node5 (平台节点)

Node2 (平台节点)

Process3 (迭代流程)

Node3 (普通节点)

平台扩展

领域扩展

节点: 全部

编码	名称	描述	节点	操作
platformAbility1	平台扩展1	能力描述	Node1	默认 扩展
platformAbility1	平台扩展1	能力描述	Node2	默认 扩展

US是数据+策略的体现，在标准能力的基础上建设“主题场景能力”，不同场景、行业组成的各式各样的页面可以由进行抽象出具体的功能节点，例如 金刚位、banner、feed流、优惠、广告、营销位、评论等等，每个功能节点是不同领域之间能力的组合，同时在不同场景和行业又可以有不同的逻辑。gbf基于业务身份（行业）和场景作为路由的key，同一个接口不同实现类来支撑不同行业 and 场景的差异，然后在Process层对各业务功能节点进行组织编排，从而形成标准的业务能力。